Natural Language Processing
DS743PE

IV B.Tech. CSE (Data Science)

Prerequisites:

Data structures and compiler design

Course Objectives:

- Introduction to some of the problems and solutions of NLP and their relation to linguistics and statistics.

Course Outcomes:

- Show sensitivity to linguistic phenomena and an ability to model them with formal grammars.
- Understand and carry out proper experimental methodology for training and evaluating empirical NLP systems
- Able to manipulate probabilities, construct statistical models over strings and trees, and estimate parameters using supervised and unsupervised training methods.
- Able to design, implement, and analyze NLP algorithms; and design different language modeling Techniques.

Syllabus

UNIT - I

Finding the Structure of Words: Words and Their Components, Issues and Challenges, Morphological Models

Finding the Structure of Documents: Introduction, Methods, Complexity of the Approaches, Performances of the Approaches, Features

UNIT - II

Syntax I: Parsing Natural Language, Treebanks: A Data-Driven Approach to Syntax, Representation of Syntactic Structure, Parsing Algorithms

UNIT – III

Syntax II: Models for Ambiguity Resolution in Parsing, Multilingual Issues

Semantic Parsing I: Introduction, Semantic Interpretation, System Paradigms, Word Sense

UNIT - IV

Semantic Parsing II: Predicate-Argument Structure, Meaning Representation Systems

UNIT - V

Language Modeling: Introduction, N-Gram Models, Language Model Evaluation, Bayesian parameter estimation, Language Model Adaptation, Language Models- class based, variable length, Bayesian topic based, Multilingual and Cross Lingual Language Modeling

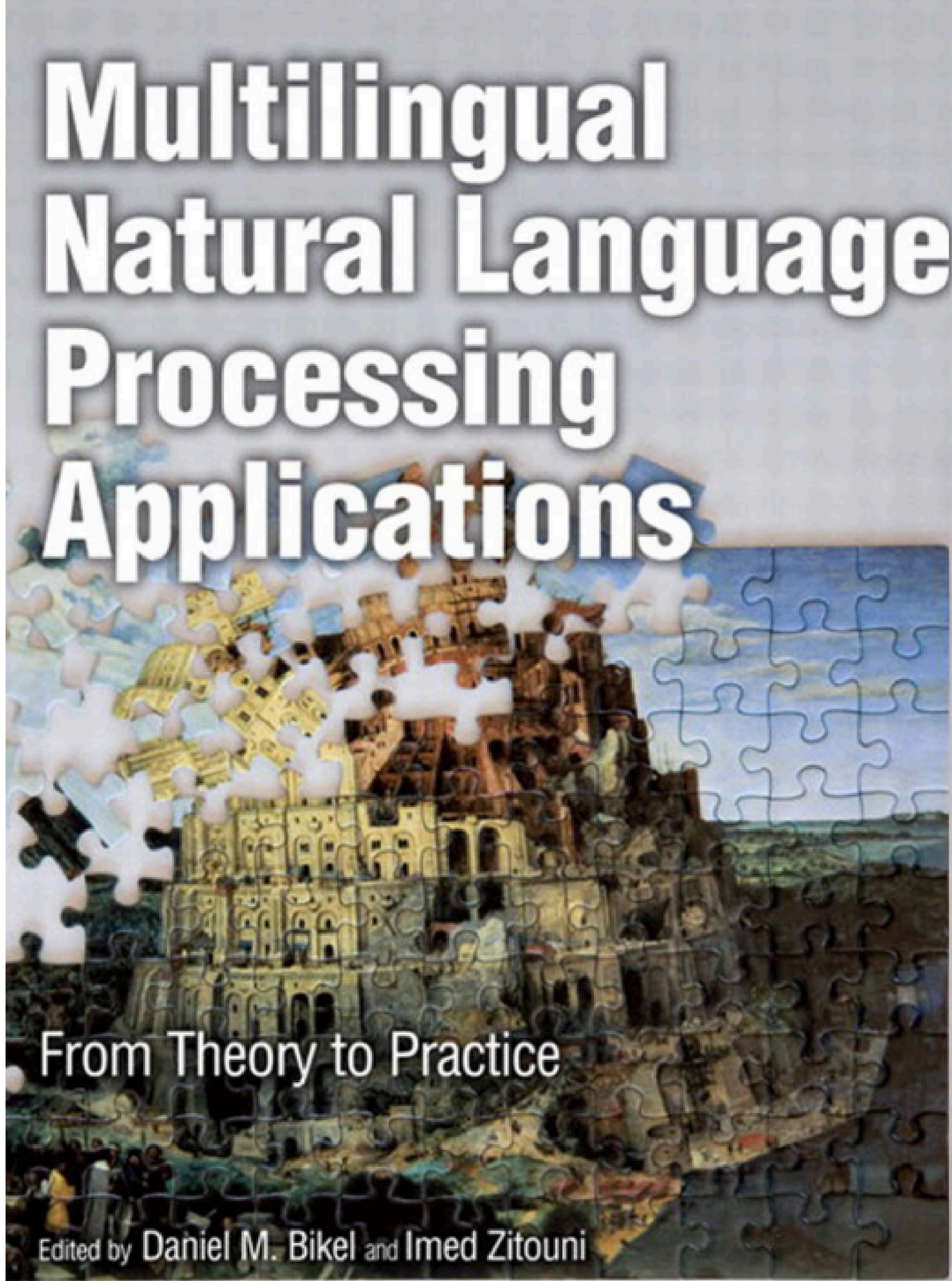
Text Book

TEXT BOOKS:

1. Multilingual natural Language Processing Applications: From Theory to Practice – Daniel M. Bikel and Imed Zitouni, Pearson Publication

REFERENCE BOOK:

1. Speech and Natural Language Processing - Daniel Jurafsky& James H Martin, Pearson Publications.
2. Natural Language Processing and Information Retrieval: Tanvier Siddiqui, U.S. Tiwary.



UNIT - I

Finding the Structure of Words: Words and Their Components, Issues and Challenges, Morphological Models

Finding the Structure of Documents: Introduction, Methods, Complexity of the Approaches, Performances of the Approaches, Features

Introduction to Language and Morphology

- Human language is a powerful tool that helps us express our thoughts, ideas, and feelings.
- We also use language to understand information from others. Language is not random; it follows certain rules and structures that help us communicate effectively.

Linguists (language experts) study language from different perspectives:

- Morphology – Study of the structure and formation of words.
- Syntax – Study of how words are arranged to form phrases and sentences.
- Phonology – Study of speech sounds and pronunciation.
- Orthography – Study of writing systems and spelling rules.
- Semantics – Study of meanings of words and sentences.
- Etymology and Lexicology – Study of the origin, history, and vocabulary of word**s.**

THE FIVE LEVELS OF LANGUAGE

Example sentence:

The cat chased the mouse.

PHONOLOGY

The sound system of language.

How the sentence sounds.

/ðə/ (the) kæt (cat) tʃeɪst (chased) ðə/ (the) maʊs/ (mouse)

MORPHOLOGY

The structure of words.

How words are formed.

The	cat	chase + -ed	the	mouse
(det)	(noun)	(verb root + past tense)	(det)	(noun)

SYNTAX

The arrangement of words.

How words are ordered to form a sentence.

The cat (Subject) → chased (Verb) → the mouse (Object)

SEMANTICS

The literal meaning of the sentence.

What the sentence means.

A cat pursued a mouse.

PRAGMATICS

The meaning in context.

What the sentence implies depending on the context.

- **Informing:** "I'm telling you what happened."
- **Answering:** Responding to "What happened?"
- **Narrating:** Part of a story or description.
- **Warning (in context):** Could imply danger or caution.

What is Morphological Parsing?

Morphological Parsing is the process of analyzing a word to identify its smaller meaningful parts (called morphemes) and understand its grammatical properties.

Example:

Word: unhappiness

un- = not

happy = root word

-ness = state or condition

So, the word unhappiness means "the state of not being happy."

Why is Morphological Parsing Important?

- Morphological parsing helps computers and humans:
- Understand the meaning of words.
- Identify grammatical information.
- Process different forms of a word.
- Improve language translation and text analysis.
- Support Natural Language Processing (NLP) applications.

Words and their Components

- Words are defined in most languages as the smallest linguistic units that can form a complete utterance by themselves.
- The minimal parts of words that deliver aspects of meaning to them are called morphemes(smallest unit that carries meaning)
    - Written symbols (letters/characters)
    - Spoken sounds (phonemes)
    - Example:
    - Unhappiness = Un + Happy + Ness
        - Un → Not
        - Happy → Root Word
        - Ness → State/Condition
-

- Depending on the means of communication, morphemes are spelled out via graphemes—symbols of writing such as letters or characters—or are realized through phonemes, the distinctive units of sound in spoken language.
- It is not always easy to decide and agree on the precise boundaries discriminating words from morphemes and from phrases

Understanding word structure helps in language analysis and Natural Language Processing (NLP).

Tokens and Tokenization

- A token is a word or meaningful unit identified in a sentence.
- Tokenization is the process of splitting text into tokens.
- Tokens are the basic units used in Natural Language Processing (NLP).

Example:
Will you read the newspaper? Will you read it? I won't read it.

Observations :
newspaper appears as a single word but can be analyzed as:
news + paper
The word won'tcan be expanded as:
will + not
Although written as one word, won't contains two separate syntactic units.

Tokenization Example

Word/Sentence	Tokenization Result	Remark
newspaper	news + paper	Compound word
won't	will + not	Contraction
Will you read the newspaper?	Will, you, read, the, newspaper	Word tokens
I won't read it.	I, will, not, read, it	Normalized tokens
నేను పాఠశాలకు వెళ్తాను	నేను, పాఠశాలకు, వెళ్తాను	Telugu word tokens
నాకు తెలుగు భాష ఇష్టం	నాకు, తెలుగు, భాష, ఇష్టం	Telugu tokenization

A token may be a complete word or a meaningful component of a word.

Tokenization helps computers understand and process natural language effectively.

- In Arabic or Hebrew [4], certain tokens are concatenated in writing with the preceding or the following ones, possibly changing their forms as well.
- The underlying lexical or syntactic units are thereby blurred into one compact string of letters and no longer appear as distinct words.
- Tokens behaving in this way can be found in various languages and are often called clitics.
- Chinese, Japanese [5], and Thai, whitespace is not used to separate words. The units that are delimited graphically in some way are sentences or clauses.
- In Korean, character strings are called eojeol ‘word segment’ and roughly correspond to speech or cognitive units, which are usually larger than words and smaller than clauses [6]

EXAMPLE 1–2: 학생들에게만 주셨는데
hak.sayng.tul.ey.key.man cwu.syess.nun.te[2]
haksayng-tul-eykey-man cwu-si-ess-nunte
student+*plural*+*dative*+only give+*honorific*+*past*+while
while (he/she) gave (it) only to the students

1. Signs used in sign languages are composed of elements denoted as phonemes, too.
2. We use the Yale romanization of the Korean script and indicate its original characters by dots. Hyphens mark morphological boundaries, and tokens are separated by plus symbols.

Lexemes

- A Lexeme is the basic dictionary form of a word.
- It represents a group of related word forms having the same core meaning.
- Lexemes form the vocabulary (lexicon) of a language.
- Different grammatical forms can belong to the same lexeme.

Lexeme	**Word Forms**
mouse	mouse, mice
see	see, sees, saw, seen, seeing
receive	receive, receiver, reception
run	run, runs, ran, running

Example 1–3: Did you see him? I didn’t see him. I didn’t see anyone.

Step 1: Tokenization

Sentence	Tokens
Did you see him?	Did, you, see, him
I didn't see him.	I, did, not, see, him
I didn't see anyone.	I, did, not, see, anyone

Step 2: Lexeme Analysis

Word Form	Lexeme
see	SEE
saw	SEE
seen	SEE
seeing	SEE

Step 3: Internal Structure of Words

Word	Possible Analysis
didn't	did + not
anyone	any + one
him	pronoun
see	verb

Morphemes and Morphs

- A Morpheme is the smallest meaningful unit in a language.
- Morphemes contribute:
    - Lexical meaning (word meaning)
    - Grammatical meaning (tense, number, etc.)
- Words can contain one or more morphemes.

Word	Morphemes	Meaning
teacher	teach + er	person who teaches
cats	cat + s	more than one cat
unhappy	un + happy	not happy
replay	re + play	play again

Morph vs Morpheme

Morph	Morpheme
Actual form appearing in a word	Abstract unit of meaning
Physical realization	Meaning/function
Example: -s	Plural morpheme

Morphemes are the building blocks of words.

Allomorphs and Morphological Variations

What are Allomorphs?

- Different forms of the same morpheme are called Allomorphs.
- The meaning remains the same.
- The form changes based on pronunciation or context.

Example: English Past Tense

Word	Past Form
walk	walked
play	played
want	wanted

Why Normalize Allomorphs?

Different Forms
↓
Normalization
↓
Single Canonical Morpheme

Word
↓
Morphs
↓
Morphemes
↓
Allomorphs
↓
Morphological Analysis

A morpheme is the smallest meaningful unit of language, while allomorphs are different forms of the same morpheme that appear in different linguistic contexts.

Example in Korean:

	concatenated		contracted		
(a)	보았-	*po-ass-*	봤-	*pwass-*	'have seen'
(b)	가지었-	*ka.ci-ess-*	가졌-	*ka.cyess-*	'have taken'
(c)	하였-	*ha-yess-*	했-	*hayss-*	'have done'
(d)	되었-	*toy-ess-*	됐-	*twayss-*	'have become'
(e)	놓았-	*noh-ass-*	놨-	*nwass-*	'have put'

- Contractions (a, b) are ordinary but require attention because two characters are reduced into one.
- Other types (c, d, e) are phonologically unpredictable, or lexically dependent.
- For example, coh-ass- 'have been good' may never be contracted, whereas noh- and ass- are merged into nwass- in (e)

Typology

- A system for classifying items into a group.
- With respect to languages: Classification of languages based on their structural families rather than their historical origin.
    - **Word Order**

SVO (Subject–Verb–Object): English – "She eats apples."
SOV (Subject–Object–Verb): Hindi, Japanese – "She apples eats."
VSO (Verb–Subject–Object): Classical Arabic, Welsh – "Eats she apples."

    - **Morphological Typology**

- **Typology vs language family**
    - Language typology groups languages by similar structure.
    - Language families group languages by common ancestry.
    - Example: English and Mandarin Chinese are not related genetically, but both are predominantly SVO languages, making them typologically similar in word order.

Morphological Typology

- Morphological typology divides languages into groups by characterizing the prevalent morphological phenomena in those languages.
- It can consider various criteria, and during the history of linguistics, different classifications have been proposed

Type	Characteristics	Examples
Isolating (Analytic)	Most words contain only one morpheme	Chinese, Vietnamese, Thai
Synthetic	Multiple morphemes are combined in a single word	Few world languages
Agglutinative	Each morpheme has a single grammatical function	Korean, Japanese, Finnish, Tamil
Fusional	One morpheme expresses multiple grammatical features	Arabic, Latin, Sanskrit, German

Types of Languages

- Isolating, or analytic, languages include no or relatively few words that would comprise more than one morpheme (typical members are Chinese, Vietnamese, and Thai; analytic tendencies are also found in English).
- Synthetic languages can combine more morphemes in one word and are further divided into agglutinative and fusional languages.
- Agglutinative languages have morphemes associated with only a single function at a time (as in Korean, Japanese, Finnish, and Tamil, etc.).
- Fusional languages are defined by their feature-per-morpheme ratio higher than one (as in Arabic, Czech, Latin, Sanskrit, German, etc.).
- Concatenative languages linking morphs and morphemes one after another.
- Nonlinear languages allowing structural components to merge non-sequentially to apply tonal morphemes or change the consonantal or vocalic templates of words.

Parsing: Issues and Challenges

- Morphological parsing tries to eliminate or alleviate the variability of word forms to provide higher-level linguistic units whose lexical and morphological properties are explicit and well defined.
- It attempts to remove unnecessary irregularity and give limits to ambiguity, both of which are present inherently in human language.
- It aims to reduce variation in word forms and resolve ambiguity.
- It helps transform complex word forms into meaningful linguistic units.

Major Challenges

1. Irregularity
   - Some words do not follow standard morphological rules.
   - Irregular forms are difficult to generalize.
2. Ambiguity
   - A word may have multiple meanings or interpretations.
   - Context is required to determine the correct meaning.
3. Productivity & Unknown Words
   - Languages continuously create new words.
   - Morphological analyzers may fail to recognize unseen words.

1. Irregularity

- Some words do not follow regular morphological patterns.
- Standard rules cannot always explain their formation.
- Rules that work for most words may fail for irregular words.
- Many irregular forms must be stored in a lexicon.

Regular Pattern	**Expected Form**	**Actual Irregular Form**	**Issue for Morphological Parser**
cat → cats	cats	cats	Easy to analyze
book → books	books	books	Easy to analyze
mouse → mouses	❌	mice	Rule fails
child → childs	❌	children	Special form must be stored
go → goed	❌	went	Completely different form
man → mans	❌	men	Internal vowel change

2.Ambiguity

- A word can have multiple meanings or grammatical interpretations.
- Morphological analyzers often require context to determine the correct interpretation.

Word/Form	**Interpretation 1**	**Interpretation 2**	**Context Needed?**
bank	Financial	River bank	Yes
bat	Flying mammal	Cricket bat	Yes
light	Illumination	Not heavy	Yes
watch	Timepiece	Observe	Yes
book	Reading material	Reserve a ticket	Yes
can	Metal container	Ability/permissio	Yes

3. Productivity

- Languages can create new words using existing morphemes.
- New words are generated continuously.
- Languages continuously create new words, many of which may not exist in the lexicon.

Base Word	**New Word Created**	**Formation Process**
read	reread	Prefixation
happy	unhappiness	Prefix + Suffix
teach	teacher	Suffixation
nation	internationalization	Multiple derivations
grandson	great-grandson	Compounding
Google	googling	Conversion/Derivation

Real-World NLP Impact

Application	Irregularity	Ambiguity	Unknown Words
Machine Translation	High	High	Medium
Chatbots	Medium	High	High
Search Engines	Low	High	High
Sentiment Analysis	Medium	High	Medium
Speech Recognition	Medium	High	High

Main Types of Morphological Models

1. Dictionary Lookup
2. Finite-State Morphology
3. Unification-Based Morphology
4. Functional Morphology
5. Morphology Induction

Morphological Models

- Morphological Models are computational methods used to analyze and generate word forms.
- They describe how words are structured and how morphemes interact.
    - Used in:
        - Natural Language Processing (NLP)
        - Machine Translation
        - Spell Checking
        - Information Retrieval
        - Text Analysis
-

1. Dictionary Lookup Approach

- Simplest morphological model.
- Every word is stored in a dictionary/database.
- Analysis is performed by searching the word in the dictionary.

Input Word
↓
Dictionary Search
↓
Retrieve Morphological Information

Word	Dictionary Entry
cats	cat + plural
children	child + plural
went	go + past tense

Advantages

✅ Easy to implement

✅ Fast lookup

✅ High accuracy for stored words

Limitations

❌ Cannot analyze unseen words

❌ Large dictionaries required

❌ Limited generalization

2.Finite-State Morphology (FSM)

- Uses Finite-State Transducers (FSTs) that encode spelling alternations.
- Represents relationships between word forms and lexical forms.
- Most popular morphological framework.

Input Word
↓
Finite State Transducer
↓
Lexical Analysis
↓
Output Features

Surface Form	**Lexical Form**
women	woman + plural
children	child + plural
replaying	re + play + ing

Advantages
✅ Efficient
✅ Fast processing
✅ Suitable for large-scale NLP systems
Limitation
❌ Difficult to model complex reduplication patterns

$$\mathcal{R} :: [\Sigma] \rightarrow \{[\Sigma]\}$$

$$\mathcal{R} :: String \rightarrow \{String\}$$

FINITE-STATE MORPHOLOGY (FST)

A finite-state transducer maps lexical representations (stems + grammatical features)
to surface word forms using states and transitions.

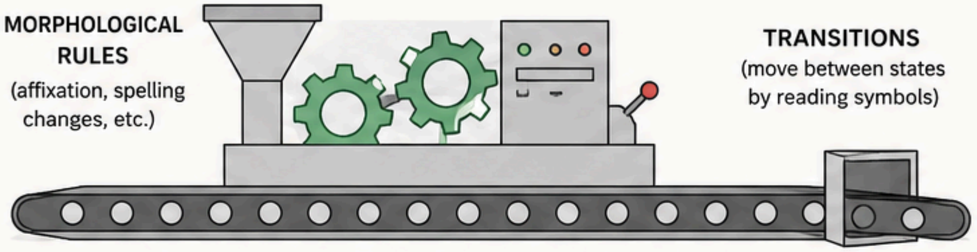
INPUT
LEXICAL FORM
(stem + features)

- walk+V+PAST
- walk+V+3SG
- walk+N+PL
- ...

FST = MORPHOLOGICAL MACHINE

MORPHOLOGICAL RULES
(affixation, spelling changes, etc.)

TRANSITIONS
(move between states by reading symbols)

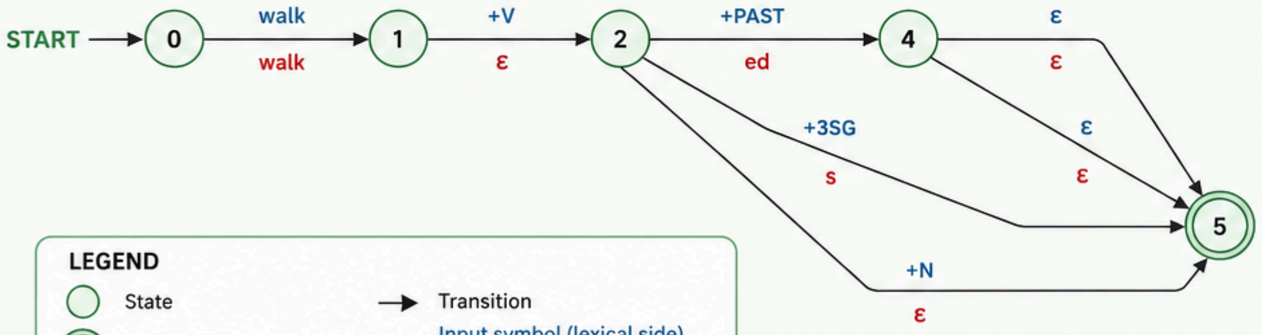


Like a machine, the FST reads the input, moves through states, applies morphological rules, and produces the correct word form.

OUTPUT
SURFACE FORM

- walked
- walks
- walks
- ...

EXAMPLE FST: VERB "WALK"



LEGEND

- State
- Accept (final) state
- Transition
- Input symbol (lexical side)
- Output symbol (surface side)
- ε = epsilon (no output)

HOW IT WORKS (TRACES)

walk+V+PAST →	walked
walk+V+3SG →	walks
walk+N+PL →	walks
walk+V+INF →	walk

WHY FST?

- Efficient and fast
- Handles complex rules and alternations
- Works for both analysis and generation
- Widely used in NLP applications

APPLICATIONS

- Morphological Analysis
- Spell Checking & Normalization
- Speech Recognition
- Machine Translation
- Information Retrieval & Search

An FST is a finite automaton with input and output on transitions. It maps one string (lexical form) to another string (surface form).

3 Unification-Based Morphology

- Uses Feature Structures instead of simple strings.
- Words are represented using grammatical features.

Word: runs

Feature	Value
Lexeme	run
POS	Verb
Person	Third
Number	Singular
Tense	Present

Feature Structure 1

+

Feature Structure 2

↓

Unified Feature Structure

Advantages

✅ Rich linguistic representation

✅ Handles complex grammar

✅ Reusable feature structures

Applications

Grammar Checking

Machine Translation

Advanced NLP Systems

UNIFICATION-BASED MORPHOLOGY

Unification-based morphology represents morphemes by feature structures (bundles of features). A word is analyzed or generated by **UNIFYING** the feature structures of morphemes so that their common information is shared and constraints are satisfied.

1. MORPHEMES AS FEATURE STRUCTURES

Each morpheme is represented by a feature structure (AVM).

walk (V stem)

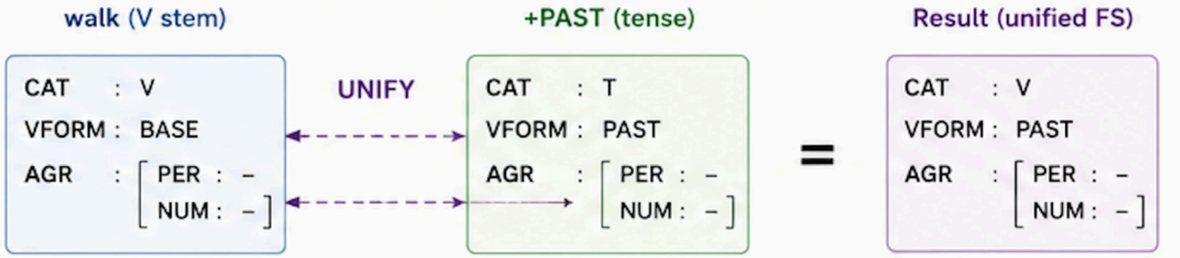
CAT	:	V
VFORM	:	BASE
AGR	:	[PER : –]
		[NUM : –]

+PAST (tense suffix)

CAT	:	T
VFORM	:	PAST
AGR	:	[PER : –]
		[NUM : –]

2. UNIFICATION (MERGING)

Common features must agree. Information from both morphemes is combined.



The unified feature structure licenses the word form **walked.**

Unification Algorithm (informal)

1. If two values are identical → keep that value.
2. If one value is a variable (e.g., – or X) → use the other value.
3. If values conflict (e.g., BASE vs PAST for VFORM) → unification fails.

3. GENERATION / ANALYSIS

The unified feature structure can be used in both directions.

Generation:

FS → Morphological rules / lexicon → Surface form

The FS above → walked

Analysis:

Surface form (walked) → Analysis rules / lexicon → Unified FS

walked → (the unified FS above)

4.Functional Morphology

- Treats morphology using mathematical functions.
- Separates linguistic knowledge from implementation.

Advantages

✅ Modular design

✅ Reusable components

✅ Easy maintenance

Main Functions

Function	**Purpose**
Parsing	Form → Meaning
Inflection	Lexeme → Word Form
Derivation	Lexeme → New Lexeme
Lookup	Meaning → Lexeme

play

↓

playing

↓

played

↓

player

$$\mathcal{I} :: lexeme \rightarrow \{parameter\} \rightarrow \{form\}$$

$$\mathcal{D} :: lexeme \rightarrow \{parameter\} \rightarrow \{lexeme\}$$

$$\mathcal{L} :: content \rightarrow \{lexeme\}$$

5.Morphology Induction

- Automatically discovers word structures from text.
- Does not require manually created dictionaries.

Example

play

plays

played

playing

Advantages

✅ Learns from data automatically

✅ Useful for low-resource languages

✅ Reduces manual effort

Challenges

❌ Ambiguity

❌ Irregular words

❌ Requires large datasets

Recap

1. Importance of human language and its five primary elements:
    a. Phonology
    b. Morphology
    c. Syntax
    d. Semantics
    e. Pragmatics
2. Important NLP perspectives of morphemes and lexemes
3. Typology and Morphological Typology
4. Morphological Parsing and its challenges
5. Parsing Methodologies

Finding the Structure of Documents:

Introduction, Methods, Complexity of the Approaches, Performances of the Approaches, Features

Outline

1. Moving from word level (morpheme) to sentence level analysis and extraction.
2. Sentence boundary detection
3. Topic segmentation
4. Statistical classification approaches for segmentation

- In human language, words and sentences do not appear randomly but usually have a structure.
  - For example, combinations of words form sentences—meaningful grammatical units, such as statements, requests, and commands.
- Likewise, in written text, sentences form paragraphs—self-contained units of discourse about a particular point or idea.
- Sentences may also be related to each other by explicit discourse connectives such as therefore.

Contents

- Sentence Boundary Detection
  - also called sentence segmentation
  - automatically segmenting a sequence of word tokens into sentence units
- Topic Boundary Detection
  - sometimes called discourse or text segmentation
  - automatically dividing a stream of text or speech into topically homogeneous blocks
  - a sequence of (written or spoken) words, the aim of topic segmentation is to find the boundaries where topics change

Sentence Boundary Detection

How does it look like in English? What challenges?

- Sentences start with a **C**apital letter and end with a period (**.**), question mark (**?**) or an exclamation (**!**) – punctuations.

“This year has been difficult for both Hertz and Avis,” said Charles Finnie, carrental industry analyst—yes, there is such a profession—at Alex. Brown & Sons.

Challenges of Sentence Boundary Detection

- Capital letters and punctuations also have other roles:
  - Capitals used for proper nouns and period, exclamation for instance as mathematical operators or abbreviations.
- Quoted sentences for instance may also have multiple sentences combined and sentence boundaries can be used within the quotes.
- SMS and instant messaging text often break grammar rules and don’t have punctuations but can still be inferred by humans.
- The more ungrammatical uttered speech or texting messages become, interannotator agreement becomes low.

SENTENCE SEGMENTATION CHALLENGES

Periods Do Double Duty

10%

of periods in the Brown Corpus mark abbreviations, not sentence ends

47%

of periods in the Wall Street Journal Corpus are abbreviations

Punctuation and case are overloaded: capital letters also mark proper nouns; periods also punctuate abbreviations and appear inside proper names.

"I spoke with Dr. Smith."
— "Dr." does not end the sentence.
"My house is on Mountain Dr." — the same abbreviation now does end it.

In this Wall Street Journal / OntoNotes sentence, only the final period ends the sentence: "...carrental industry analyst—yes, there is such a profession—at Alex. Brown & Sons."

Code-Switching

- Code-switching: multilingual speakers mix words, phrases, or sentences from multiple languages.
- Punctuation conventions can travel with either language:
  - The writer may keep the first language's punctuation rules, or
  - Switch to the second language's convention — e.g., Spanish precedes questions with an inverted “¿”
- Technical text redefines punctuation meaning too: URLs, programming languages, and mathematical notation all reuse periods, slashes, and symbols.
- These special constructs must be detected and parsed explicitly for adequate processing of technical text.

Hand Written Rules vs Classifier

Rule-Based Systems

- Patterns + lists of known abbreviations disambiguate boundaries (e.g., don't split after “Mr.” or “Gov.”)
- Fail on unknown abbreviations, sentence-final abbreviations, and typos
- Not robust to non-standard text (forums, chats, blogs) or to speech, which has no typographic cues at all
- Every language needs its own rule set

Classification Approach

- Train a classifier on human-annotated boundary data instead of hand-written rules
- Text: only candidates at punctuation marks are classified as boundary / not
- Speech: every word boundary is a candidate, since typographic cues are absent

Topic Segmentation

- Also called **discourse segmentation** or text segmentation.
- Given a sequence of written or spoken words, the goal is to find the boundaries where topics change.
- A classic example is a broadcast news program, where each story is a topically coherent block separated from the next by a topic boundary.
- Applications:
  - **Information retrieval** — return only the segment relevant to a user's query instead of a whole long document
  - **Automatic summarization** — summarize coherent units rather than an undifferentiated stream

Topic Segmentation

Tens of thousands of people are homeless in northern China tonight after a powerful earthquake hit an earthquake registering 6.2 on the Richter scale at least 47 people are dead. Few pictures available from the region but we do know temperatures there will be very cold tonight -7 degrees. <TOPIC_CHANGE> Peace talks expected to resume on Monday in Belfast, Northern Ireland. . . .

Figure 2–1. Example of a topic boundary in a news article

Early Topic Segmentation Systems: DARPA

- During the late 1990s, the U.S. Defense Advanced Research Projects Agency (DARPA) initiated the Topic Detection and Tracking (TDT) program
- One core TDT task: segmenting a continuous news stream into individual stories.
- TDT established a common test bed; researchers also build simulated environments, e.g., by concatenating Reuters news stories.

The Trouble with Defining “Topic”

- Topic segmentation is a nontrivial problem **without a very high human agreement** because of many **natural-language-related issues** and hence requires a good definition of topic categories and their granularities.
- Topics are **not flat** — they occur in a semantic hierarchy, so granularity is inherently ambiguous.
- Example: a sentence about soccer followed by one about baseball — one annotator marks a topic change, another doesn't, since both fall under “sports.”

Other Challenges for Topic Segmentation: Difference in Modalities

Text

- Explicit segmentation cues: headlines
- Paragraph breaks mark structure directly
- Typographic punctuation for sentence ends

Speech

- No headlines or paragraph breaks available
- Pause duration signals a likely break
- Speaker changes can flag a boundary

This mirrors the same text-vs-speech divide seen in sentence segmentation: typography for text, timing and voice for speech.

Segmentation Methods

- Think of sentences/topics as **boundaries.**
- We first obtain “**candidates**“, each candidate has a *<start>* and *<end>* word to mark the start and end of each boundary.
- Each candidate boundary has a **feature vector** $x \in X$ (the observation) and a **label** $y \in Y$ to predict.
- Example features: word n-grams around the candidate, an “inside a quotation” flag, abbreviation-list membership, or pause/pitch/energy for speech.
- Binary case: y = b (boundary) or non-boundary.
- Given training pairs {x, y}, learn a function that assigns accurate labels to unseen candidates.

Segmentation Methods

- Defined as a **probability task:** finding the most probable sentence or topic boundaries.
- The natural unit of sentence segmentation is words and of topic segmentation is sentences

y is list of candidate boundaries and x is samples/text

$$\hat{y}_i = \underset{y_i in \mathcal{Y}}{\operatorname{argmax}} P(y_i|\mathbf{x}_i)$$

this is called **local** segmentation

- The words or sentences are then grouped into contiguous stretches belonging to one sentence or topic—that is, the word or sentence boundaries are classified into sentence or topic boundaries and nonboundaries.

Local vs Sequence Classification

single label

Local Modeling

Each candidate is classified in isolation — estimate the single most probable label for each example independently.

Sequence Modeling

Search for the entire label sequence with maximum joint probability across all candidates together.

multiple labels

Why dependencies matter: in broadcast news speech, two consecutive boundaries that form a one-word sentence are very rare — a real dependency sequence models can exploit.

A second, independent axis: generative methods model the joint distribution P(X,Y); discriminative methods model the differences between labelings directly.

1. Generative Sequence Classification Model

Hidden Markov Model

label/candidate vector feature vector

$$\widehat{Y} = \underset{Y}{\operatorname{argmax}}\, P(Y|X) = \underset{Y}{\operatorname{argmax}}\, \frac{P(X|Y)P(Y)}{P(X)} = \underset{Y}{\operatorname{argmax}}\, P(X|Y)P(Y)$$

denominator can be dropped
because its fixed for any given y∈Y

- If the size of Y is m, then its a m-state markov model. Topic boundary problems use multiple boundaries while sentence boundary is binary.
- The most probable boundary sequence is obtained by dynamic programming, thanks to the Viterbi algorithm that is used for decoding Markov models

n-gram model

$$P(Y) = \prod_{i=1}^{n} P(y_i|y_1, \ldots y_{i-1})$$

each individual y here represents one topic

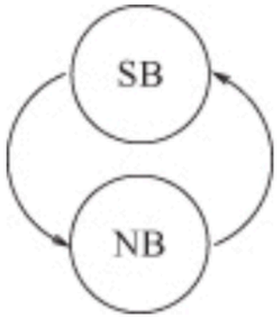
bigram model

$$P(y_i|y_1, \ldots, y_{i-1}) \approx P(y_i|y_{i-1})$$

Markov Models

$P(y_i|y_{i-1})$ - state transition probability
$P(x_i|y_i)$ - state observation likelihood

- In the case of n-gram models, we tend to have increased complexities when calculating state transition probabilities.



SB- Sentence Boundary
NB- Non-boundary

Figure 2–2. Conceptual hidden Markov model for segmentation with two states: one for segment boundaries, one for others

Emitted Words	...	people	are	dead	few	pictures	...
State Sequence	...	NB	NB	SB	NB	NB	...

Extending the HMM: HELM & fHELM

- Hidden Event Language Model (HELM) — Shriberg et al. — grew out of earlier work on modeling speech disfluencies (Stolcke & Shriberg).
- Boundary tokens become extra **“meta-tokens,”** each emitted by its own dedicated state.
- These extra boundary tokens are then used to capture other **meta-information**.
- The most commonly used meta-information is the feedback obtained from other classifiers.
- Typically, the **posterior probability** of being in that boundary state is used as a state observation likelihood after being divided by prior probabilities
- Factored HELM (fHELM) — Guz et al. — adds **morphological/POS information**, inspired by factored language models.

2.Discriminative Local Classification Model

- **Model P(y|x) directly**, instead of assuming a generative class density.
- Each boundary is classified separately with **local and contextual features**; **no global** (sentence- or document-wide) optimization is performed.
- A wide range of algorithms have been applied to sentence segmentation: transformation-based learning, regression trees, neural networks, C4.5 decision trees, maximum entropy, support vector machines, and naive Bayes.
- Mikheev treated sentence segmentation as a POS-tagging subtask — assigning a tag to punctuation like any other token — using a combined HMM + maximum entropy approach.
- Discriminative methods often outperform generative ones, at the cost of iterative, multi-pass training.

TextTiling: Lexical Cohesion

- A lexical cohesion metric is used in a word vector space as an indicator of topic similarity.
- TextTiling can be seen as a local classification method with a single feature of similarity.
- The document is “chopped” wherever similarity between neighboring blocks drops below a threshold.
- **Blocks** are compared where adjacent blocks are seen to how many words they have similar to one other – imagine capturing improtant keywords for each topic this way. This similarity is given by the following formula

$$\frac{\sum_t w_{t,b_1} \cdot w_{t,b_2}}{\sqrt{\sum_t w_{t,b_1}^2 \sum_t w_{t,b_2}^2}}$$

w_t is the weight assigned to term t in block b. Blocks b_1 and b_2 are compared

3. **Discriminative Sequence Classification Methods**

- In segmentation tasks, the sentence or topic decision for a given example (word, sentence, paragraph) highly depends on the decision for the examples in **its vicinity**.
- Sequence decisions are interdependent, so discriminative sequence models add a **decoding stage** on top of local classifiers.
- **Conditional Random Fields (CRFs)** are the most widely used: a **log-linear model** over the whole label sequence, conditioned on the whole observation sequence.
- Trained by maximizing (regularized) likelihood via gradient, conjugate-gradient, or online methods.
- Viterbi decoding again finds the most probable label sequence at test time.

Discriminative Sequence Classification Methods

- Formally, they model the conditional probability of a sequence of boundary labels ($Y = y_1,\ldots, y_n$) given the sequence of feature sets extracted from the context in which they occur ($X = x_1,\ldots, x_n$).
- CRFs have been successful across many sequence-labeling tasks, including sentence segmentation in speech.

$$P(Y|X) \sim \frac{1}{Z(X)} \exp\left(\sum_{t=1}^{n}\sum_{i=1}^{m} \lambda_i f_i(y_{t-1}, y_t, y_t)\right)$$

$$Z(X) = \sum_{Y} \exp\left(\sum_{t=1}^{n}\sum_{i=1}^{m} \lambda_i f_i(y_{t-1}, y_t, y_t)\right)$$

$f_i(\cdot)$ are feature functions of the observations and a clique of labels, and λ_i are the corresponding weights. $Z(\cdot)$ is a normalization function dependent only on the observations.

Challenges and Advantages

- In terms of complexity, training of discriminative approaches is more complex than training of generative ones because they require multiple passes over the training data to adjust for their feature weights.
- Generative models such as HELMs can handle multiple orders of magnitude larger training sets.
- Discriminative classifiers allow for a wider variety of features and perform better on smaller training sets.

Hybrid Approaches

- The problem: nonsequential discriminative classifiers ignore context, and real-valued features (pause, pitch) don't fit cleanly without binning.
- New idea: take posterior probabilities from a strong discriminative classifier (boosting, CRF, ...) and convert them into HMM state-observation likelihoods by dividing out the prior probabilities (Bayes' rule).
- Feed these into Viterbi decoding over the HMM/HELM for the final segmentation; a weighting scheme (α, β, tuned on held-out data) balances transition vs. observation probabilities.

Features for Text and Speech

Lexical Features for Topic Segmentation

Lexical Cues:

The presence of certain cue phrases represented as *c* in sentences will be represented as lexical features for performing topic segmentation.

Similarity:

A second type of feature is the similarity of the content before and after the boundary, typically expressed as the cosine similarity between the previous and next sentences.

Lexical Chains:

A lexical chain is a sequence of words in a document that are semantically related (same word, synonym, or words from the same topic (for example, leaf, rose, flower)). Then, for a candidate boundary between w_i and w_{i+1}, the broken-lexical-chain feature can be computed as:

l = a word before the boundary,
k = a word after the boundary

set of lexical chains

$$x_{\text{chain}} = \left| \left\{ c \in \mathcal{C} : \min_{\substack{w_k, w_l \in c \times c \\ k \leq i, l > i}} l - k > d_{\min} \right\} \right|$$

one lexical chain

wi is index of each word

Features for Text and Speech

Syntactic & Discourse Features

Syntactic

- For morphologically rich languages, such as Czech and Turkish, morphological analyses of words are used as additional cues.
- POS / morphological tag n-grams around the boundary, mirroring the word n-gram features

Discourse

- Disource is the relation between sentences and are at a higher level than syntactic features.
- Change of speaker may indicate a sentence boundary, and commercials may indicate a topic boundary in broadcast news or conversations.

Prosodic Features for Speech

Pause

start(w_{i+1}) − end(w_i). The simplest, most reliable feature; often thresholded around ~0.2s.

Pitch (F0)

Look for a reset or fall across the boundary; undefined in unvoiced regions.

Duration

Pre-boundary lengthening — the final rhyme stretches before a boundary.

Energy & Voice Quality

Fall near boundaries too, but noisier, harder to normalize, and least successful of the cues.

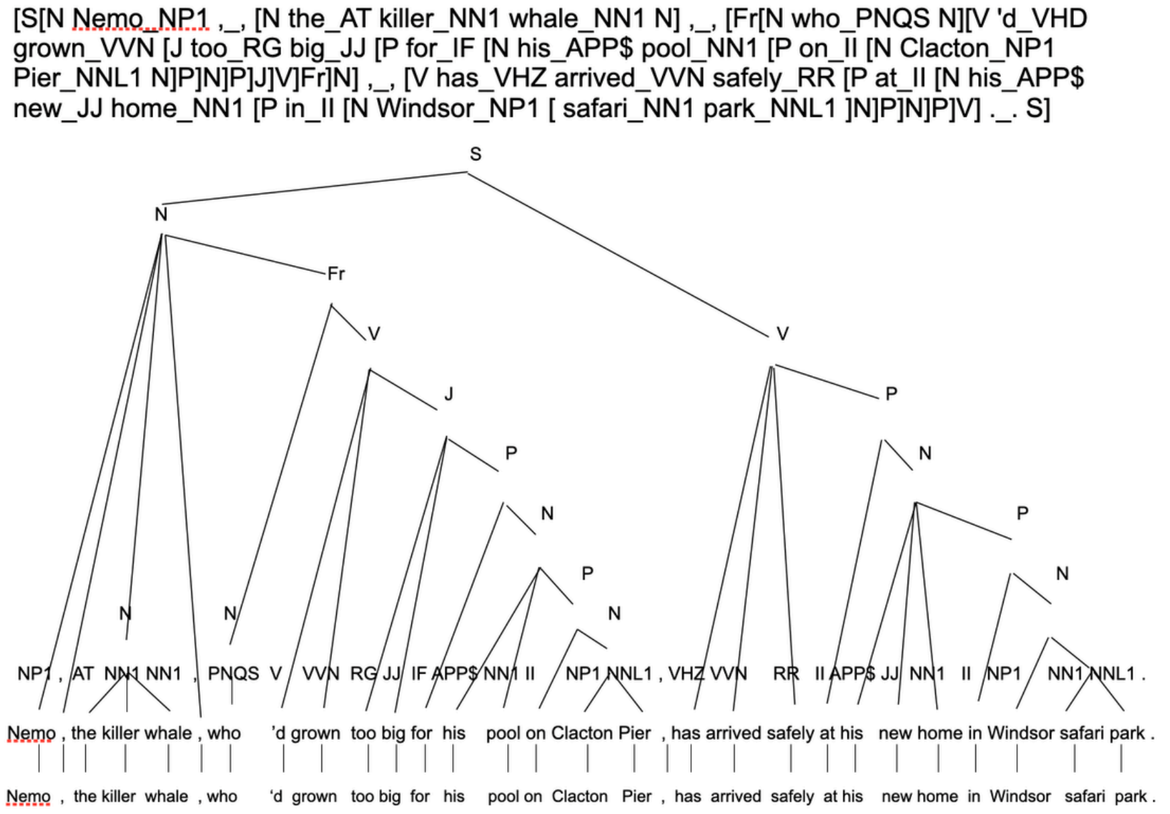
Topic boundaries show the same cues, amplified: longer pauses, an extra-high F0 reset, wider pitch/intensity range, and speaking-rate shifts — perceptible even in filtered, unintelligible speech. Galley et al. found speaker activity, silence/overlap, and cue phrases improve topic-shift detection; Hsueh, Moore & Renals found the gain holds for coarse shifts but not fine-grained ones.

UNIT - II

Syntax I: Parsing Natural Language, Treebanks: A Data-Driven Approach to Syntax, Representation of Syntactic Structure, Parsing Algorithms

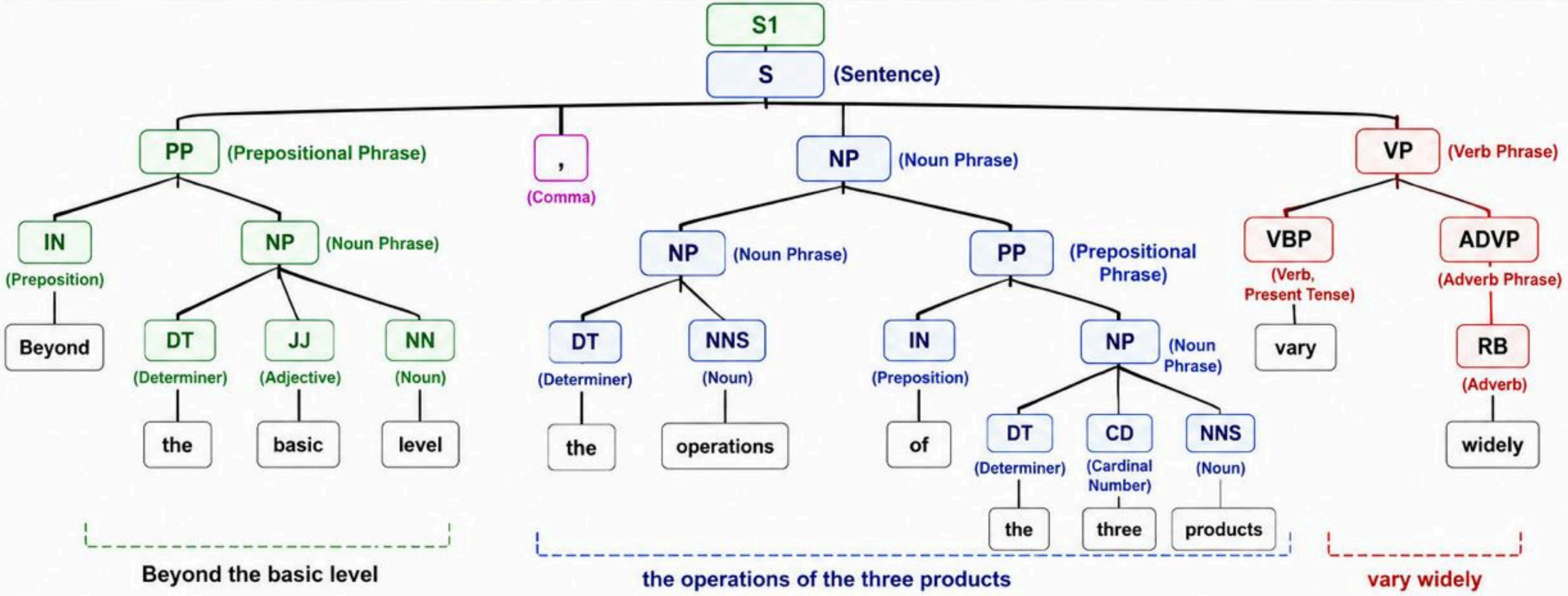
Syntax & Parsing

- Parsing uncovers the hidden structure of linguistic input.
- In natural language processing (NLP), the **syntactic analysis** of natural language input can vary from being very low-level, such as simply tagging each word in the sentence with a **part of speech (POS),** or very high level.
- Example of high-level: recovering a **structural analysis** that identifies the **dependency** between each predicate in the sentence and its explicit and implicit arguments.
- POS tags give information about the individual words, and their internal form (eg singular vs plural, tense of verb)



Example of parsing

Sentence: Beyond the basic level, the operations of the three products vary widely.



LEGEND (Tags)

S – Sentence	DT – Determiner	VBP – Verb (Present Tense)
PP – Prepositional Phrase	JJ – Adjective	RB – Adverb
NP – Noun Phrase	NN – Noun (Singular)	ADVP – Adverb Phrase
VP – Verb Phrase	NNS – Noun (Plural)	
IN – Preposition	CD – Cardinal Number	

How the sentence is compressed:

- ✓ Remove PP (Beyond the basic level)
- ✓ Remove Comma (,)
- ✓ Remove CD (three)
- ✓ Remove ADVP (widely)

Compressed Sentence:

The operations of the products vary.

Key Point: The parse tree shows the grammatical structure of a sentence. By identifying optional parts (circled in the original diagram) that do not change the core meaning, we can generate a shorter but meaningful sentence.

Why Parse?

He wanted to go for a drive in movie.

vs

He wanted to go for a drive in the country.

- There is a natural pause between the words *drive* and *in* in sentence 2 that reflects an underlying hidden structure to the sentence.
- Parsing can provide a structural description that identifies such a break in the intonation.

Paraphrasing

open borders imply increasing racial fragmentation in EUROPEAN COUNTRIES

open borders imply increasing racial fragmentation in the countries of europe.

open borders imply increasing racial fragmentation in europe.

- These are interchangable without changing meaning of sentence.
- Paraphrasing models are built on top of parsers, which do two jobs here:
  - Identify the target constituent that is eligible to be replaced
  - Find appropriate replacement phrases that can legally substitute for it
- Paraphrases of this type have proven useful in **statistical machine translation.**

Parsers in Applications

Statistical machine translation
Structure guides how phrases move between languages

Information extraction
Pull relations and facts out of large text collections

Summarization
Compress and select at the constituent level

Language generation
Produce entity grids that keep generated text coherent

Error correction in text
Detect what is structurally wrong, not just misspelled

Knowledge acquisition
Discover semantic classes and ‘x is-a y’ relationships

Speech recognition
Syntax as a language model — valuable for disfluent, error-prone speech

Dialog & text-to-speech systems
Interpretation, and natural-sounding intonation

Context Free Grammar

- Parsing recovers information that is not explicit in the input sentence.
- This implies that a parser requires some knowledge in addition to the input sentence about the kind of syntactic analysis that should be produced as output.
- Rules can be written down for instance using **context-free grammar (CFG).**

CFG:

- Words are the **terminal** symbols. They are introduced by rules of the form $X \rightarrow w$, where X is the part of speech of word w.
  - These **POS nonterminals** are called **part-of-speech tags** or **preterminals**
  - Example: the rule $V \rightarrow$ 'bought' has POS symbol V generating the verb bought
- Parsing with a CFG then means deriving the input sentence from the grammar's start symbol — and the derivation is the parse tree.

Context Free Grammar

Example of a CFG

```
S -> NP VP
NP -> 'John' | 'pockets' | D N | NP PP
VP -> V NP | VP PP
V -> 'bought'
D -> 'a'
N -> 'shirt'
PP -> P NP
P -> 'with'
```

NP means noun phrase,
VP is a verb phrase
N is a noun,
V is a verb etc.
Phrase constitutes of more than a word,
for example with an adjective

- Here, we have created these set of “rules” that can help us parse a sentence easily.
- When we see “John pockets” in a new sentence, we will know that that would be a NP.
- When we see any praposition (P) with a noun phrase (NP), that is considered a PP

Context Free Grammar

“John bought a shirt with pockets”

```
(S (NP John)
   (VP (V bought)
       (NP (NP (D a)
               (N shirt))
           (PP (P with)
               (NP pockets)))))
```

```
S -> NP VP
NP -> 'John' | 'pockets' | D N | NP PP
VP -> V NP | VP PP
V -> 'bought'
D -> 'a'
N -> 'shirt'
PP -> P NP
P -> 'with'
```

Context Free Grammar

```
(S (NP John)
   (VP (V bought)
       (NP (NP (D a)
               (N shirt))
           (PP (P with)
               (NP pockets)))))
```

Pockets are the currency used to buy the shirt.

vs

```
(S (NP John)
   (VP (VP (V bought)
           (NP (D a)
               (N shirt)))
       (PP (P with)
           (NP pockets))))
```

John buys a shirt that has pockets.

✓ more plausible

Context Free Grammar

Sample Problem:

"The dog chased the cat with a stick."

CFG:

```
S  -> NP VP
NP -> D N | NP PP
VP -> V NP | VP PP
PP -> P NP
D  -> 'the' | 'a'
N  -> 'dog' | 'cat' | 'stick'
V  -> 'chased'
P  -> 'with'
```

(S (NP (D the) (N dog))
(VP (VP (V chased)
(NP (D the) (N cat)))
(PP (P with)
(NP (D a) (N stick)))))

The dog “uses” a stick
for the chasing

(S (NP (D the) (N dog))
(VP (V chased)
(NP (NP (D the) (N cat))
(PP (P with)
(NP (D a) (N stick))))))

The cat has a stick

Problems with writing a CFG

- Unlike a programming language**, natural language** is far too complex.
- A simple list of rules does not consider interactions between **different components** in the grammar.
- Listing all possible **syntactic constructions** in a language is a difficult task.
- **Knowledge acquisition problem:** it is difficult to exhaustively list **lexical properties** of words, for instance, to list all the grammar rules in which a particular word can be a participant.
- Recursive rules produce ambiguity:

```
N -> N N
N -> 'natural' | 'language' | 'processing' | 'book'
```

natural → one parse
natural language → one parse (rule used once)
natural language processing → two parses
natural language processing book → five parses

Problems with writing a CFG

- For CFGs it can be proved that the number of parses obtained by using the recursive rule n times is the Catalan number of n:

$$\mathrm{Cat}(n) = \frac{1}{n+1}\binom{2n}{n}$$

- This poses a serious computational problem for parsers. For an input with n words, the number of possible parses is exponential in n.
- For most natural language tasks, we do not wish to explore this entire space of ambiguity, as computationally take $O(n^3)$ time complexity and with storage complexity of n^2 .

How Treebanks help

- A treebank is simply a **collection of sentences** (also called a corpus of text), where each sentence is provided a complete syntax analysis.
- A style book or set of **annotation guidelines** is typically written before a human annotation process to ensure a **consistent scheme of annotation** throughout the treebank.
- Crucially, a treebank contains **no explicit grammar**. There is no rule list, and typically no inventory of syntactic constructions.
  - No exhaustive set of rules is even assumed to exist
  - **Assumptions about syntax** are present, but they are implicit in the annotations
- Instead of rules, a detailed annotation guideline (a style book) is written before annotation begins, so human experts produce a single, consistent, most-plausible analysis for each sentence.
- Consistency is measured, not assumed: roughly 10% of the material is annotated by more than one annotator, and **interannotator agreement** is computed on that overlap.

Two Knowledge-Acquisition Problems solved by Treebanks

Problem 1: What are the rules?

- The syntactic analysis is given directly, so the grammar underlying it never has to be written down.
- A parser needs no explicit rules at all — it only needs to faithfully produce an analysis. The knowledge it learns can be seen as an implicit set of rules.

Problem 2: Which parse is right?

- Because every sentence carries its most plausible analysis, supervised learning can be used to learn a scoring function over all possible analyses.
- The parser mimics the human annotator's decisions — using the input and its own previous decisions as evidence.

Syntax Analysis used for Treebanks

Treebanks are built with one of two representations.

Dependency Graphs

- Words are the only nodes. Directed arcs link a head to its dependents.
- Minimal assumptions — no hidden structure, no empty elements, no extra hierarchy.
- Favoured for freer word-order languages (e.g. Czech, Turkish), where a predicate's arguments turn up in many different positions.

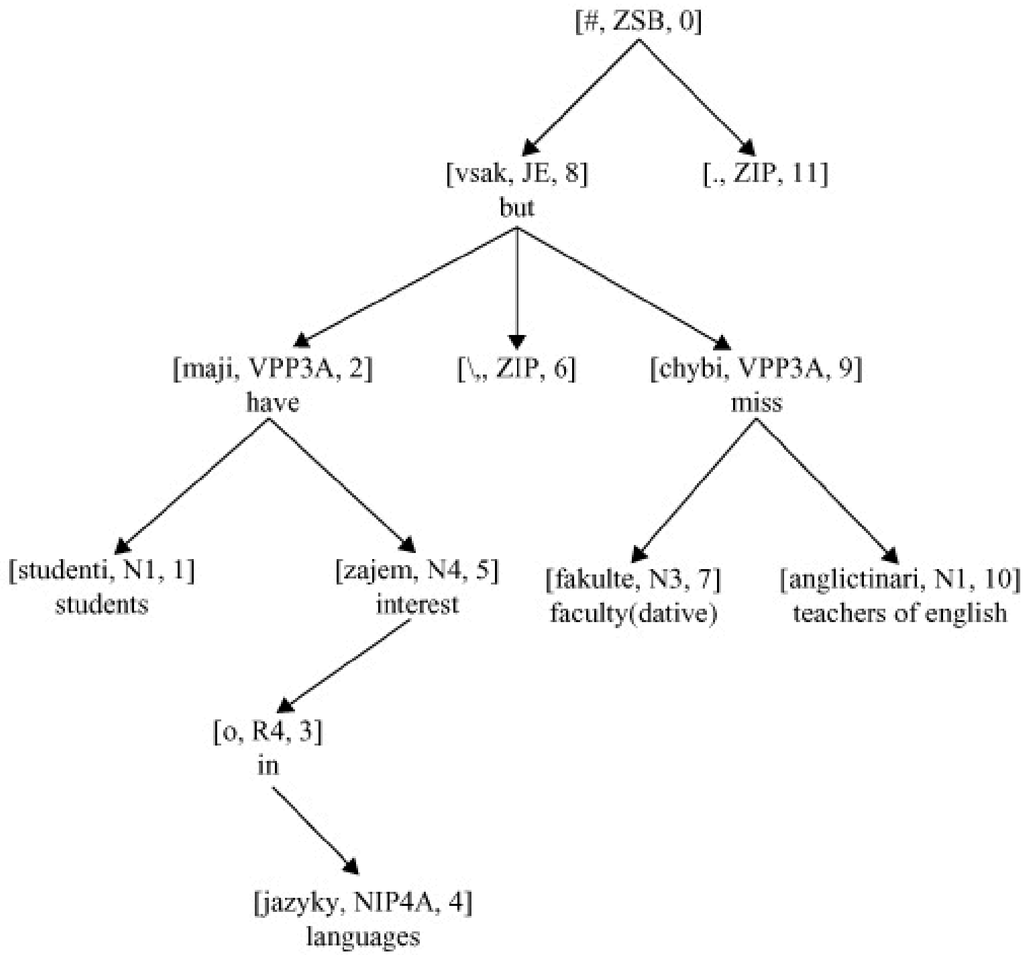
Phrase Structure Trees

- Partition the sentence into constituents; merge smaller constituents into larger ones.
- Adds information about long-distance dependencies, using null elements as placeholders.
- Typically used for less flexible word-order languages (e.g. English, French).

Representation of Syntactic Structure

Syntax Analysis Using Dependency Graphs

- Each node in the graph is a word, its part of speech, and the position of the word in the sentence
- The philosophy: connect a word — the head of a phrase — with the dependents in that phrase, using a directed (and therefore asymmetric) link.



Dependency Graphs: The Core Idea

- The philosophy: connect a word — the **head of a phrase** — with the **dependents** in that phrase, using a **directed (and therefore asymmetric) link**.
- The **head–dependent relation** can be read either **semantically (head–modifier)** or **syntactically (head–specifier)**. Like phrase structure, the notation is compatible with many linguistic frameworks.
- The defining difference from phrase structure: dependency analyses make **minimal assumptions about structure**. They deliberately avoid:
  - Annotation of **hidden structure**
  - Empty elements standing in for missing or displaced arguments
  - Any hierarchical structure that is not strictly necessary
- The words of the input sentence are the only vertices in the graph. Nothing else exists in the analysis.

Dependency Graphs

- There are many variants of dependency syntactic analysis, but the basic textual format for a dependency tree can be written in the following form, where each dependent word specifies the head word in the sentence, and exactly one word is dependent to the root of the sentence.

Index	Word	Part of Speech	Head	Label
1	They	PRP	2	SBJ
2	persuaded	VBD	0	ROOT
3	Mr.	NNP	4	NMOD
4	Trotter	NNP	2	IOBJ
5	to	TO	6	VMOD
6	take	VB	2	OBJ
7	it	PRP	6	OBJ
8	back	RB	6	PRT
9	.	.	2	P

Dependency Trees: Projectivity

- **Projectivity** is a constraint imposed by the linear order of words on the dependencies between words.
- A **projective dependency tree** is one where if we put the words in a linear order based on the sentence with the root symbol in the first position, the dependency arcs can be drawn above the words without any crossing dependencies.
- **Language dependent:** English has very few cases in a treebank that will need a nonprojective analysis. In other languages, such Czech and Turkish, the number of nonprojective dependencies can be much higher.

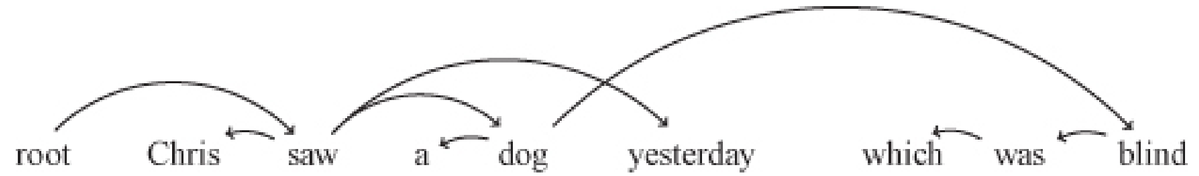
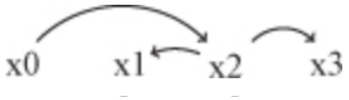
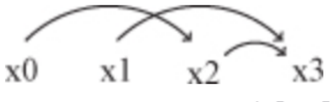


Figure 3–3. An unlabeled nonprojective dependency tree with a crossing dependency

Projectivity and CFGs



→ **Projective ✅** → Can be converted to a CFG and vice-versa



→ **Non Projective ❌** → CFG not possible

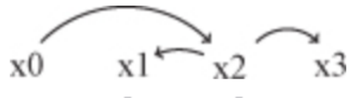
Example CFG

```
X0_2 -> X0_1* X2_1
X0_1 -> x0*
X2_1 -> X1_1 X2_2*
X1_1 -> x1*
X2_2 -> X2_3* X3_1
X2_3 -> x2*
X3_1 -> x3*
```

where x0, x1, x2, x3 are non-terminals
* represents a dependency
X0_{1,2}, ... X3_{1,2} are terminals

this can be converted to

Resulting projective graph



Projectivity and CFGs

- There is no CFG that can capture the nonprojective dependency.
- Projectivity definition: for each word in the sentence, its descendants form a contiguous substring of the sentence.
- a nonprojective dependency means that there is a word in the sentence such that its descendants do not form a contiguous substring of the sentence.
- If we want a dependency parser to only produce projective dependencies, we can implicitly create an equivalent CFG that will ignore all nonprojective dependencies.

Phrase Structure Trees

- A phrase structure syntax analysis of a sentence derives from the traditional sentence diagrams that partition a sentence into constituents, and larger constituents are formed by merging smaller ones.
- A phrase structure tree can be viewed as implicitly having a **predicate-argument structure** associated with it.
  - Example: the following phrase structure analysis shows that the subject of the sentence is marked with **-SBJ marker** and the predicate is marked with the **-PRD marker**.

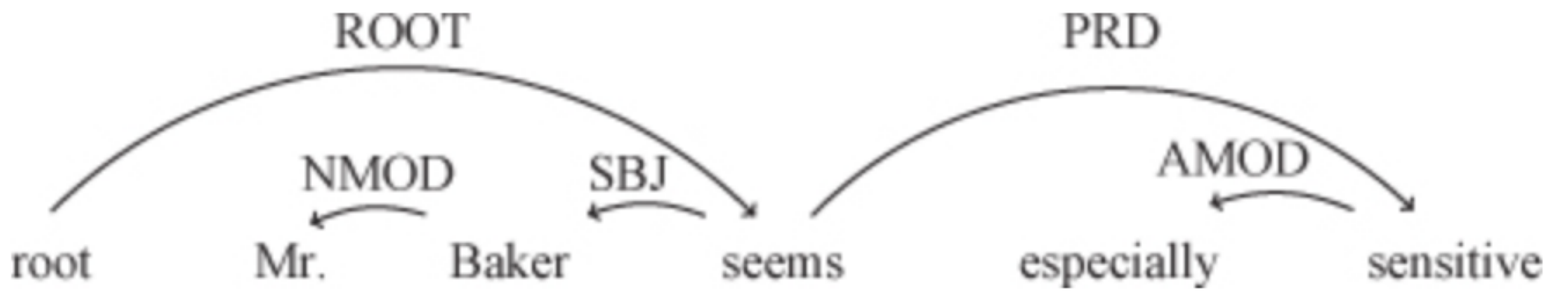
```
(S (NP-SBJ (NNP Mr.)
           (NNP Baker))
   (VP (VBZ seems)
       (ADJP-PRD (RB especially)
                 (JJ sensitive))))

Predicate-argument structure:
seems((especially(sensitive))(Mr. Baker))
```

Mr. Baker seems especially sensitive

Phrase Structure Trees

- A phrase structure tree can be read as implicitly carrying a predicate–argument structure, which is recovered from the labels on the tree.
- Function labels do this work. For example a subject may be marked -SBJ, a predicate -PRD, a logical subject -LGS — letting you read the underlying relations straight off the tree.
- This is the fundamental trade against dependency: phrase structure records more, including things that are not visible in the string — at the cost of a heavier, more theory-laden annotation.



Equivalent dependency tree for - **Mr. Baker seems especially sensitive**

- Some of the information carried by the bracketing labels reappears as labels on the dependency arcs.
- But a dependency analysis would typically not link the subject directly to the predicate — doing so would create an inconvenient crossing dependency with the arc from seems to the root.

Null Elements: When not all constituents are present

```
(SBARQ (WHNP-1 What)
       (SQ is (NP-SBJ Tim)
              (VP eating (NP *T*-1)))
       ?)

Predicate-argument structure:
eat(Tim, what)
```

Example Sentence: What is Tim eating?

- We can easily define the **SUBJ** in the sentence as “Tim”, but the *object* is not apparent. Hence, we use a ***T*** marker to mark the presence of a *non-apparent* **null** object.
- A null element is a constituent with no yield — the same idea as an epsilon in formal language theory. It has no words, but it localizes a predicate–argument dependency that the surface string has displaced.

```
(S (NP-SBJ (PRP They))
   (VP (VP (VBD persuaded)
           (NP-1 (NNP Mr.)
                 (NNP Trotter))
           (S (NP-SBJ (-NONE- *-1))
              (VP (TO to)
                  (VP (VB take)
                      (NP (PRP it))
                      (PRT (RB back))))))))

Predicate argument structure:
persuade(they, Mr. Trotter, take_back(Mr. Trotter, it))
```

The missing subject is the OBJECT of persuaded.

```
(S (NP-SBJ-1 (PRP They))
   (VP (VP (VBD promised)
           (NP (NNP Mr.)
               (NNP Trotter))
           (S (NP-SBJ (-NONE- *-1))
              (VP (TO to)
                  (VP (VB take)
                      (NP (PRP it))
                      (PRT (RB back))))))))

Predicate argument structure:
promise(they, Mr. Trotter, take_back(they, it))
```

The missing subject is the SUBJECT of promised.

- **Dependency analysis,** however, cannot see this difference. The dependency trees for these two sentences are identical — same words, same heads, same labels. The distinction lives entirely in the null element.

Modern Parsers using Phrase Structure Banks

- Most statistical parsers trained on phrase structure treebanks simply ignore the difference.
- The rich annotation — logical subjects, null elements, function tags — is all but ignored during training. The parser learns the bracketing, not the empty categories.
- Using post-processing we can recover empty elements and identifies their antecedents.
- Dedicated evaluation schemes compare parsers specifically on recovery of the underlying predicate–argument structure.
- The lesson for a practitioner: the treebank may contain far more information than your parser is actually using. Knowing what is in there tells you what you could be exploiting.

Parsing Algorithms

- Given an input sentence, how do you actually search a space of exponentially many analyses — and come back with the right one in polynomial time?
- Given an input sentence, a parser produces an output analysis of that sentence, which we now assume is the analysis that is consistent with a treebank used to train a parser.
- Treebank parsers do not need to have an explicit grammar, but to make the explanation of parsing algorithms simpler we first consider parsing algorithms that assume the existence of a CFG.

Parsing Algorithms

```
N -> N 'and' N
N -> N 'or' N
N -> 'a' | 'b' | 'c'
```

Example CFG

```
N
=> N 'or' N
=> N 'or c'
=> N 'and' N 'or c'
=> N 'and b or c'
=> 'a and b or c'
```

Example of a Derivation of above CFG

- A **derivation** is a sequence of rule applications leading from the start symbol to the input string.
- Each line of a derivation is a **sentential form**.

Parsing Algorithms

```
'a and b or c'
=> N 'and b or c'          # use rule N -> a
=> N 'and' N 'or c'        # use rule N -> b
=> N 'or c'                # use rule N -> N and N
=> N 'or' N                # use rule N -> c
=> N                       # use rule N -> N or N
```

- **Rightmost derivation:** In this derivation, we only expand on the rightmost nonterminal in each sentential form.
- An interesting property of a rightmost derivation is revealed if we arrange the derivation in *reverse order*.
- To build a parser, we need to create an algorithm that can perform the steps in the preceding rightmost derivation for any grammar and for any input string.

Parsing Algorithms

For this derivation:

```
N
=> N 'or' N
=> N 'or c'
=> N 'and' N 'or c'
=> N 'and b or c'
=> 'a and b or c'
```

We can have **one** parse tree

```
(N (N (N a)
      and
      (N b))
   or
   (N c))
```

But for a parse tree:

```
(N (N (N a)
      and
      (N b))
   or
   (N c))
```

We can have many different derivations:

```
'a and b or c'
=> N 'and b or c'      # use rule N -> a
=> N 'and' N 'or c'    # use rule N -> b
=> N 'and' N 'or' N    # use rule N -> c
=> N 'and' N           # use rule N -> N or N
=> N                   # use rule N -> N and N
```

Shift Reduce Parsers

1. Start with an **empty stack** and the **buffer** containing the input string.
2. Exit with success if the top of the stack contains the start symbol of the grammar and if the buffer is empty.
3. Choose between the following two steps (if the choice is ambiguous, choose one based on an oracle):
   - Shift a symbol from the buffer onto the stack.
   - If the top k symbols of the stack are $\alpha_1 \ldots \alpha_k$, which corresponds to the righthand side of a CFG rule $A \rightarrow \alpha_1 \ldots \alpha_k$, then replace the top k symbols with the left-hand side nonterminal A.
4. Exit with failure if no action can be taken in previous step.
5. Else, go to step 2.

Shift Reduce Parsers

Operations

- **Shift:** Move the next input symbol onto the stack when no reduction is possible.
- **Reduce:** Replace a sequence of symbols at the top of the stack with the left-hand side of a grammar rule.
- **Accept:** Successfully complete parsing when the entire input is processed and the stack contains only the start symbol.
- **Error:** Handle unexpected or invalid input when no shift or reduce action is possible.

Shift Reduce Parsers

Example: CFG

```
N -> N 'and' N
N -> N 'or' N
N -> 'a' | 'b' | 'c'
```

- Here the list of terminals are Σ = {a, b, c, and, or}
- First we construct a “formal language theory”-equivalent input sentence buffer by converting list of terminals into a sentence.
- Example **Input Buffer**: a and b or c

Shift Reduce Parsers

Parse Tree	Stack	Input	Action
		a and b or c	Init
a	a	and b or c	shift a
(N a)	N	and b or c	reduce `N -> a`
(N a) and	N and	b or c	shift and
(N a) and b	N and b	or c	shift b
(N a) and (N b)	N and N	or c	reduce `N -> b`
(N (N a) and (N b))	N	or c	reduce `N -> a`
(N (N a) and (N b)) or	N or	c	shift or
(N (N a) and (N b)) or c	N or c		shift c
(N (N a) and (N b)) or (N c)	N or N		reduce `N -> c`
(N (N (N a) and (N b)) or (N c))	N		reduce `N -> N or N`
(N (N (N a) and (N b)) or (N c))	N		Accept!

Shift Reduce Parsers

- Shift-reduce parsing allows a linear time parse but requires access to an oracle (in our case CFG).
- For general CFGs in the worst case, such a parser might have to resort to **backtracking** (happens when no other operations possible), which means reparsing the input, which leads to *exponential time* in the worst case.
- Worst Case: $O(n^3)$ where n is the length of the input.

CKY Hypergraph Parser

- In order to avoid backtracking, we can first **binarize** the CFG by allowing **atmost two symbols** at the right per rule.
- Hence,

```
N -> N 'and' N
N -> N 'or' N
N -> 'a' | 'b' | 'c'
```

```
N -> N N^
N^ -> 'and' N
N -> N Nv
Nv -> 'or' N
N -> 'a' | 'b' | 'c'
```

(two non-terminals N^, Nv are introduced)

- We can specialize any CFG G_c to a particular input string by creating a new CFG that represents a compact encoding of all possible parse trees that are valid in grammar G_c for just this particular input sentence.
- Label the gaps between words with positions called **spans**:
  - `0 a 1 and 2 b 3 or 4 c 5`

CKY Hypergraph Parser

- The nonterminals in this **forest grammar** G_c include the span information. The different parse trees that can be generated using this grammar are the valid parse trees for the input sentence.

```
N -> N N^
N^ -> 'and' N
N -> N Nv
Nv -> 'or' N
N -> 'a' | 'b' | 'c'
```

```
N[0,5] -> N[0,1] N^[1,5]
N[0,3] -> N[0,1] N^[1,3]
N^[1,3] -> 'and'[1,2] N[2,3]
N^[1,5] -> 'and'[1,2] N[2,5]
N[0,5] -> N[0,3] Nv[3,5]
N[2,5] -> N[2,3] Nv[3,5]
Nv[3,5] -> 'or'[3,4] N[4,5]
N[0,1] -> 'a'[0,1]
N[2,3] -> 'b'[2,3]
N[4,5] -> 'c'[4,5]
```

CKY Hypergraph Parser

```
N[0,5] -> N[0,1] N^[1,5]
N[0,3] -> N[0,1] N^[1,3]
N^[1,3] -> 'and'[1,2] N[2,3]
N^[1,5] -> 'and'[1,2] N[2,5]
N[0,5] -> N[0,3] Nv[3,5]
N[2,5] -> N[2,3] Nv[3,5]
Nv[3,5] -> 'or'[3,4] N[4,5]
N[0,1] -> 'a'[0,1]
N[2,3] -> 'b'[2,3]
N[4,5] -> 'c'[4,5]
```

- This span-annotated grammar is a compact encoding of all legal parse trees at once, called a "parse forest."
- Notice N[0,5] appears on the left of two rules: those two rules are the two ambiguous readings, sharing sub-parts.
- When you let nodes with the same label+span be shared/merged instead of duplicated, the forest becomes a hypergraph

CKY Hypergraph Parser

Pseudocode

```
for i = 0 ... n:
if there is a rule N -> x (score s) for the
    token x spanning [i, i+1]:
  add N[i, i+1] -> x[i, i+1] with score s
```

- These rules can be constructed by simply checking for the existence of rules of the type N → x (examples given on the right) for any input token x and creating a specialized rule for token x.

N[0,1] -> 'a'[0,1]
N[2,3] -> 'b'[2,3]
N[4,5] -> 'c'[4,5]

CKY Hypergraph Parser

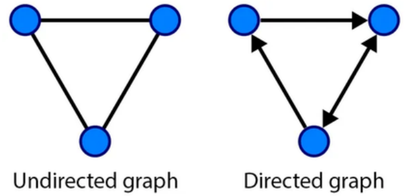
- The CKY algorithm considers every span of every length and splits up that span in every possible way to see if a CFG rule can be used to derive the span.
- Eventually we are guaranteed to find a rule that spans the entire input string if such a rule exists.
- Examining the loop structure of the algorithm shows that it takes n^3 time for an input of size n.
- However, exhaustively listing all trees from the specialized CFG will take exponential time in the worst case.
- However, picking the most likely tree by using supervised machine learning will take no more than n^3 time.

Probabilistic Way to do this

- If scores are log probabilities, the best-scoring tree is the Viterbi parse.
- For instance, we can compare the score of X[i, j] with the score of the *current highest scoring* entry Y [i, j] and throw away any rule starting with X[i, j] if it is extremely low when compared to Y [i, j].
- In this way we can trade off accuracy for a much faster parsing time. This is called **Beam Thresholding**.
- Beam thresholding risks a search error (missing the true best tree) but that's rare.
- **Global thresholding** — delete a chunk if it has no possible *neighbors* to combine with; it can never be part of a full parse, so it's dead weight. This looks across the whole chart, not just one span.

Minimum Spanning Trees (MST)

- Finding the optimum branching in a directed graph is closely related to the problem of finding a minimum spanning tree in an undirected graph.
- The directed graph case is of interest because it corresponds to a dependency tree, which is always *rooted* and cannot have *cycles*.
- Why this matters for parsing: a dependency tree is inherently directional. The word "saw" is the head and "John" is its dependent — the arrow points from head to dependent, saw → John.

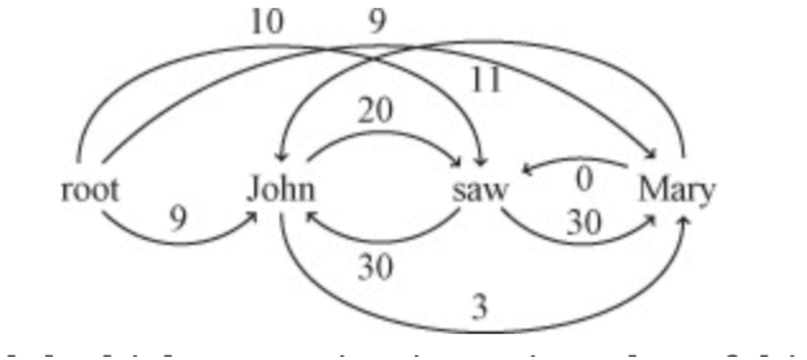


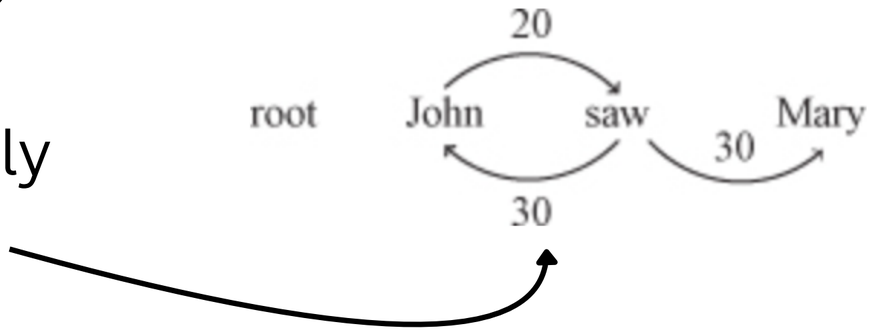
Minimum Spanning Trees (MST)

- Instead of the CKY/chart machinery from before, we use every possible dependency arrow between every pair of words with a possible **score** — a **fully connected directed graph.**
- Now the parsing problem becomes: pick a subset of arrows that forms a valid dependency tree and has the highest total score.
- The MST formulation ignores word order. It only looks at which word points to which, not whether the arrows cross when you draw them above the sentence. This lets it recover non-projective (crossing) dependencies.

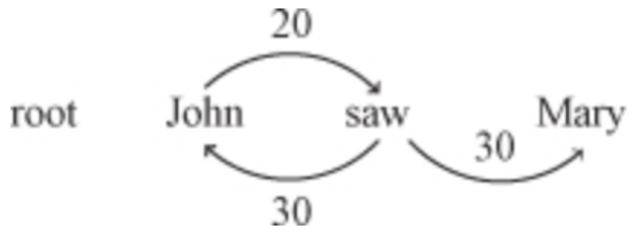
Minimum Spanning Trees (MST)

- Ex sentence: *John saw Mary*
- The first step is to find the highest scoring incoming edge.
- If this step results in a tree, then we report this as the parse because it would have to be an MST.
- In this example, however, after we pick only the highest scoring incoming edges from the graph, we do have a cycle.

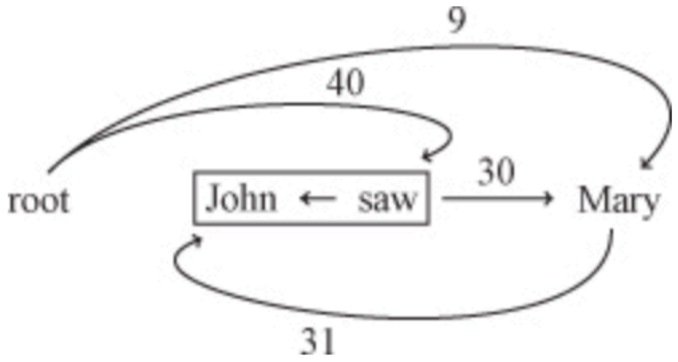




Minimum Spanning Trees (MST)



- **Contract the cycle** - When we compute the edge weights from each node to that contracted node, we also have to keep track of which component of the merged node is the one with maximum weight. For example, in the preceding graph, we compare the incoming edge: **root->saw->John** with **root->John->saw** and compare the incoming edge: **Mary->John->saw** with **Mary->saw->John** to obtain the ones with maximum weight shown below:



- **Recurse-** Now run the exact same algorithm on this smaller graph (with the contracted node). Keep contracting cycles until you get a graph with no cycles.

Models for Ambiguity Resolution in Parsing

- Last section answered "how do I search for the best tree efficiently".
- This answers "how do I score a tree so that the best-scoring one is actually the correct one?"
- The algorithms from that section are used by the models described in this section to find the highest scoring parse tree or dependency analysis and sometimes to train the models as well.

Probabilistic Context-Free Grammars

```
(S (NP John)
   (VP (VP (V bought)
           (NP (D a)
               (N shirt)))
       (PP (P with)
           (NP pockets))))
```

```
(S (NP John)
   (VP (V bought)
       (NP (NP (D a)
               (N shirt))
           (PP (P with)
               (NP pockets)))))
```

- We want to provide a model that matches the intuition that the second tree above is preferred over the first.
- We can add scores or probabilities to the rules in this CFG in order to provide a score or probability for each derivation.
- The probability of a derivation is the sum of scores or product of probabilities of all the CFG rules used in that derivation.

Probabilistic Context-Free Grammars

- **The core idea -** Take a plain CFG and attach a probability to every rule. Now every parse tree gets a probability = the product of the probabilities of all the rules used to build it (or, in log space, the sum of scores).
- When a sentence has multiple parses, you pick the one with the highest probability. A CFG with probabilities on its rules is a PCFG.

```
S -> NP VP (1.0)
NP -> 'John' (0.1) | 'pockets' (0.1) | D N (0.3) | NP PP (0.5)
VP -> V NP (0.9) | VP PP (0.1)
V -> 'bought' (1.0)
D -> 'a' (1.0)
N -> 'shirt' (1.0)
PP -> P NP (1.0)
P -> 'with' (1.0)
```

Probabilistic Context-Free Grammars

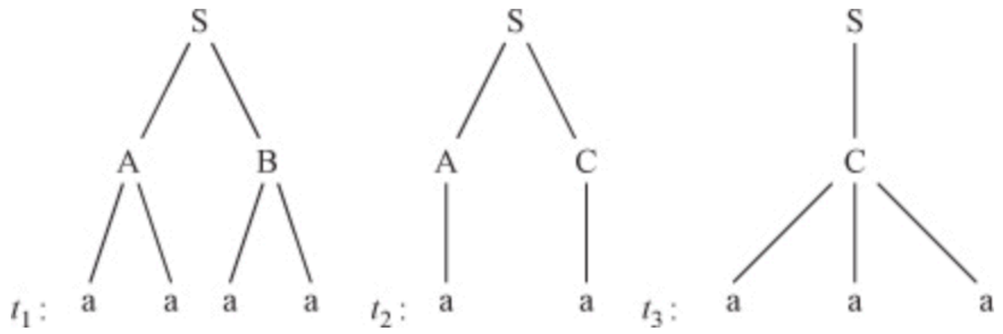
- Because scores can be viewed simply as log probabilities, we use the term probabilistic context-free grammar (PCFG) when scores or probabilities are assigned to CFG rules.
- For the probabilities over all possible trees to be a valid probability distribution, each rule's probability is conditioned on its left-hand side: for a rule N → α, you use P(N → α | N).
- Concretely, all the rules that expand a given non-terminal must have probabilities that sum to 1.
- Look at their example: NP → John (0.1) | pockets (0.1) | D N (0.3) | NP PP (0.5) — those add to 1.0.

$$1 = \sum_{\alpha} P(N \rightarrow \alpha)$$

Probabilistic Context-Free Grammars

- "John bought a shirt with pockets." Two readings:
  - Reading 1: "with pockets" attaches to the verb phrase, as if John used pockets to do the buying. (Nonsense, but grammatically legal.) This uses the rule VP → VP PP.
  - Reading 2: "with pockets" attaches to the noun "shirt", the shirt has pockets. (The sensible reading.) This uses the rule NP → NP PP.
- From these rule probabilities, the only deciding factor for choosing between the two parses for John bought a shirt with pockets is the two rules NP -> NP PP and VP -> VP PP because all the other rules in one parse also occur in the other.
- **The well-formedness condition -** For the probabilities over all possible trees to be a valid probability distribution, each rule's probability is conditioned on its left-hand side: for a rule N → α, you use P(N → α | N). Concretely, all the rules that expand a given non-terminal must have probabilities that sum to 1.

Probabilistic Context-Free Grammars



- The rule probabilities can be derived from a **treebank**, as we can observe from the following example. Consider a treebank with three trees t_1, t_2, and t_3.
- If we assume that tree t_1 occurred 10 times in the treebank, t_2 occurred 20 times, and t_3 occurred 50 times, then the PCFG we obtain from this treebank would be:

$$\frac{10}{10+20+50} = 0.125 \quad S \rightarrow A\ B$$
$$\frac{20}{10+20+50} = 0.25 \quad S \rightarrow A\ C$$
$$\frac{50}{10+20+50} = 0.625 \quad S \rightarrow C$$
$$\frac{10}{10+20} = 0.334 \quad A \rightarrow a\ a$$
$$\frac{20}{10+20} = 0.667 \quad A \rightarrow a$$
$$\frac{20}{20+50} = 0.285 \quad B \rightarrow a\ a$$
$$\frac{50}{20+50} = 0.714 \quad C \rightarrow a\ a\ a$$

Probabilistic Context-Free Grammars

- PCFGs are **context-free**: once you decide to expand a non-terminal, any rule with that non-terminal on the left is allowed, and the rule's probability doesn't depend on where in the tree you are or what's around it. So the model can freely mix-and-match rule applications into combinations that were never observed together.
- Sometimes that's good (*generalization*); often it produces *implausible parses* because the independence assumption is too strong.
- The fix: enrich the labels. To break these bad independence assumptions, statistical parsers make the non-terminal labels carry more context.

Probabilistic Context-Free Grammars

- **Parent annotation** - staple each node's parent label onto it. A VP under an S becomes VP^S, distinct from a VP under another VP. Now the rule probabilities can differ by context, because the labels are the context.
- **Automatic split-merge with EM** - instead of hand-designing the splits, let an *unsupervised algorithm* discover useful subcategories of each non-terminal. It splits labels into latent sub-labels and merges back the ones that didn't help, using the **Expectation-Maximization algorithm**.
- **Lexicalization** - attach the actual head word to each non-terminal. So an NP isn't just "NP," it's "NP-headed-by-shirt." This is powerful because attachment decisions really depend on the specific words: "shirt with pockets" vs. "bought with cash" attach differently because of the words involved, not the phrase categories.

Generative Models for Parsing

- To find the most plausible parse tree, the parser has to choose between the possible derivations each of which can be represented as a sequence of decisions.
- Building a parse tree = making a sequence of decisions, $D = d_1, d_2, \ldots, d_n$. Each d_i is one choice (which rule to apply, which word to attach, etc.). The probability of the whole tree is the product of the probability of each decision given all the decisions before it: $P(d_i \mid d_1, \ldots, d_{i-1})$.
- **History** - That conditioning context $d_1, \ldots, d_{i-1}$ — called the history — represents the partially-built tree so far. But the history keeps growing and is different for every sentence, so you can't estimate $P(d_i \mid \text{history})$ directly; there are infinitely many possible histories ; hence needs to be shrunk.

Generative Models for Parsing

- **Solution?** Introduce a function Φ that maps each history into a fixed, finite set of features $\varphi_1(H_i), \ldots, \varphi_k(H_i)$.
- In other words, you don't remember the entire history and you summarize it with a handful of features (e.g., "the parent label," "the head word," "the previous sibling").
- Two different histories that produce the same features are treated as equivalent; grouped into the same equivalence class.
- This is "generative" because it models how the tree is generated step by step, and the whole thing is essentially a dressed-up PCFG.
- **Limitation?** Because it's built on PCFG-style independence assumptions, a generative model is constrained in *which* features it can use; the CFG structure dictates the conditioning. You often want to score a parse using *arbitrary features* of the whole tree (any property you can dream up, not just local rule context). Discriminative models are designed exactly for this freedom.

Discriminative Models for Parsing

- The **global linear model** : This is a clean, general framework that covers parsing, chunking, and tagging all at once.
  - **x** = an **input** (a sentence). y = a candidate output (a parse tree, a tag sequence, a dependency analysis).
  - **Φ(x, y)** = a **feature vector**: a big list of numbers summarizing properties of the pair (x, y). Any feature you want — "does this tree contain the rule NP→NP PP?", "is the head word 'shirt'?", "does an arrow cross another?" — anything.
  - **w** = a **weight vector**, one weight per feature, learned from data. It says how much each feature matters.
  - **Score** of a candidate = **Φ(x, y) · w** (the dot product — multiply each feature by its weight and sum). Higher = more plausible.
  - **GEN(x)** = the function that produces all candidate outputs for input x (e.g., all valid parse trees

Discriminative Models for Parsing

$$F(x) = \underset{y \in GEN(x)}{\operatorname{argmax}} \; p(y \mid x, \mathbf{w})$$

- F(x) returns the highest scoring output y* from GEN(x).
- In plain words: generate all candidates, score each with the dot product, output the highest-scoring one. The word "global" means the score is over the entire structure at once, not decision-by-decision.
- **CRF vs. the global linear model:** A Conditional Random Field (CRF) turns that linear score into a proper conditional probability by adding a *normalization term* whereas global linear model just drops the normalization and uses the raw score.

Multilingual Issues

- So far we have assumed that in a grammar or in a treebank, the notion of a word, or more specifically that of a word token, is well defined; but only for a *given treebank*.
- Everything so far assumed the input arrives as a neat sequence of words. For most of the world's languages that assumption is false (and the ways it fails are exactly the ways parsers break when ported to a new language).
- Example:
  - In English, a token is typically separated from other tokens by a space character.
  - However, a parser treats *today's* as two tokens (*today* and *'s*), and *There's* as *There* and *'s*.

Multilingual Issues

- And the possessive marker does not even attach to the previous token — it can apply to an entire preceding constituent:

```
(NP (NP (NP First)
        (PP of
            (NP America))
        's)        ──────→ s is a whole constituent here
    operating results)
```

Similarly for the copula verb *'s*:

```
(S (NP-SBJ (EX There))
   (VP (VBZ 's)
       (NP-PRD (NP (NN nothing))
               (ADJP (RB very)
                     (JJ hot))))))
```

- Example 1: The point this example is making: the 's is not attached to just the nearest word "America." It sits as a sibling of the whole inner phrase "First of America," meaning the possession is "[First of America]'s operating results" — the operating results belong to the entire company name, not to "America."
- Example 2: The sentence is "There's nothing very hot" (meaning "There is nothing very hot"). Here 's is the verb "is," not a possessive.

Multilingual Issues

Issue 1: Case

- In some languages there are issues with **uppercase** and **lowercase**.
  - For **standaridization** purposes**,** It is tempting to lowercase all your treebank data and simply lowercase input texts for the parser.
  - **Why standardize?** a word at the start of a sentence is capitalized just because it's first, not because it's special, so you might selectively lowercase sentence-initial tokens to get cleaner counts.
  - However, case can carry useful information. The token *Boeing* if it is previously unseen in the training data might look like a progressive verb like *singing ((the -ing ending)*, but the initial uppercase character makes it more likely to be a proper noun.

Multilingual Issues

- Solution? Replace rare words like *Patagonia* with a case-preserving pattern: if Patagonia appears only twice, replace it with a pattern like Xxx (capital, then lowercase letters).
- Then any new unknown word matching that shape is treated as the same known "pattern token." Same idea for dates, times, IP addresses, URLs - collapse them into pattern tokens so the parser isn't derailed by never-before-seen exact strings.

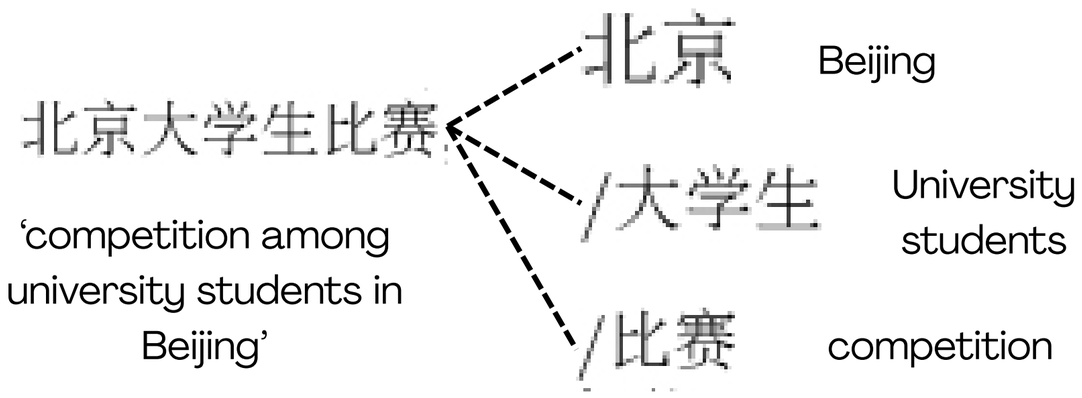
Multilingual Issues

Issue 2: Encoding

- Text isn't just characters, it's bytes in some encoding. If your treebank is in one encoding and your input text is in another, you must convert one to match the other.
- Some languages, such as Chinese, are encoded in different formats depending on the place where the text originates—for example, GB, BIG5, and UTF-8.
- This is however a trivial issue, algorithmically, compared to writing a parser.

Multilingual Issues: Word Segmentation

- Many writing systems, Chinese especially, put characters side by side with no spaces marking word boundaries. So before you can tag or parse, you must decide where one word ends and the next begins. And this choice changes the meaning.
-



- However, 北京大学 can also mean “Beijing University” and if we segment this way the resulting word will be “‘Beijing University gives birth to competition’”.

Multilingual Issues: Word Segmentation

- Word segmentation is the process of demarcating blocks in a character sequence such that the produced output is composed of separated tokens and is *meaningful*.
- Why it matters for parsing: POS tags (NNP, DT, ...) attach to words, and the syntax tree is built over words, so nothing downstream works until segmentation is done.

Approach 1

- One interesting approach to **Chinese parsing** is to parse the character sequence directly.
- So, instead of segmenting first and parsing second, the Chinese parser parses straight over the character sequence, and lets the tree itself define words: a non-terminal that spans a group of characters effectively declares "these characters form a word."

Multilingual Issues: Word Segmentation

- This study found that **immediate local context** was most useful for predicting word boundaries; the global sentence context (the full parse) helped surprisingly little — even though occasionally segmentation ambiguity really does need long-distance information from the tree.

Approach 2

- Feed the parser a lattice of options. The danger of a pipeline (segment once → then parse) is that the parser is stuck with a single segmentation and can't reconsider it even if it leads to a terrible parse.
- A CFG parser can parse an entire word lattice — a finite-state automaton encoding many possible segmentations at once. The parser uses the automaton's states as its indices (generalizing the position indices 0,1,2,... we used for spans in CKY).

Multilingual Issues: Word Segmentation

- So instead of committing to one segmentation, you hand the parser several ranked segmentations bundled in a lattice, and it picks whichever segmentation yields the best parse. Segmentation and parsing get decided together rather than in a lossy pipeline.

Multilingual Issues: Morphology

- In many languages a single "word" is built from several pieces called morphemes, a stem plus attached grammatical bits : and the word's meaning is the combination of those pieces.
- So a word isn't atomic; it's stem + morphemes, and the syntax can depend on the morphemes inside a word, not just the word as a whole.
- **Agglutinative languages (Turkish, Finnish, ...)** take this to the extreme: an entire clause's worth of meaning can be packed into one long word by stacking morphemes.
- **Inflectional languages (Czech, Russian)** are less extreme: morphemes mark *case*, *gender*, *degree*, *polarity*, and these dimensions are orthogonal — they combine independently.

Multilingual Issues: Morphology

Solution?

- **Turn morphology into (complex) POS tagging**: Rather than physically splitting each word into morphemes, you tag each word with a rich, structured POS tag that encodes all its morphological dimensions at once.
- **Morphology-aware parser:** A morphology-aware parser can use agreement to disambiguate.